



MIAGE- L3

Systèmes Informatiques

Système d'exploitation

Gestion de la mémoire

BAKAYOKO Ibrahima

Enseignant-chercheur en informatique

UFR Mathématiques & Informatique

Bakayoko.ibrahima1@ufhb.edu.ci

Open Our Mind for a free world

La gestion de la mémoire est l'un des piliers fondamentaux de l'informatique et de l'ingénierie logicielle. La mémoire joue un rôle central dans le fonctionnement des systèmes informatiques, car elle est responsable du stockage et de la gestion des données et des programmes en cours d'exécution.

Imaginez un ordinateur comme une bibliothèque immense, où chaque livre représente une partie de votre programme ou de vos données. La gestion de la mémoire, c'est comme l'organisation et le contrôle de cette bibliothèque. Si elle est mal gérée, cela peut entraîner des problèmes graves tels que des plantages de logiciels, des fuites de mémoire, des ralentissements du système, et bien plus encore.

Objectif général

- Permettre aux étudiants d'acquérir une compréhension approfondie des concepts, des techniques et des bonnes pratiques liés à la gestion de la mémoire dans le domaine de l'informatique et du développement logiciel.

Objectifs spécifiques

- Comprendre les fondements de la gestion de la mémoire;
- Maîtriser les concepts clés;
- Explorer les stratégies de gestion de la mémoire;
- Éviter les problèmes courants

PLAN

INTRODUCTION

I. MÉMOIRE VIRTUELLE

II. REMPLACEMENT DES PAGES

III. ALLOCATIONS DE MÉMOIRE

CONCLUSION

MÉMOIRE VIRTUELLE

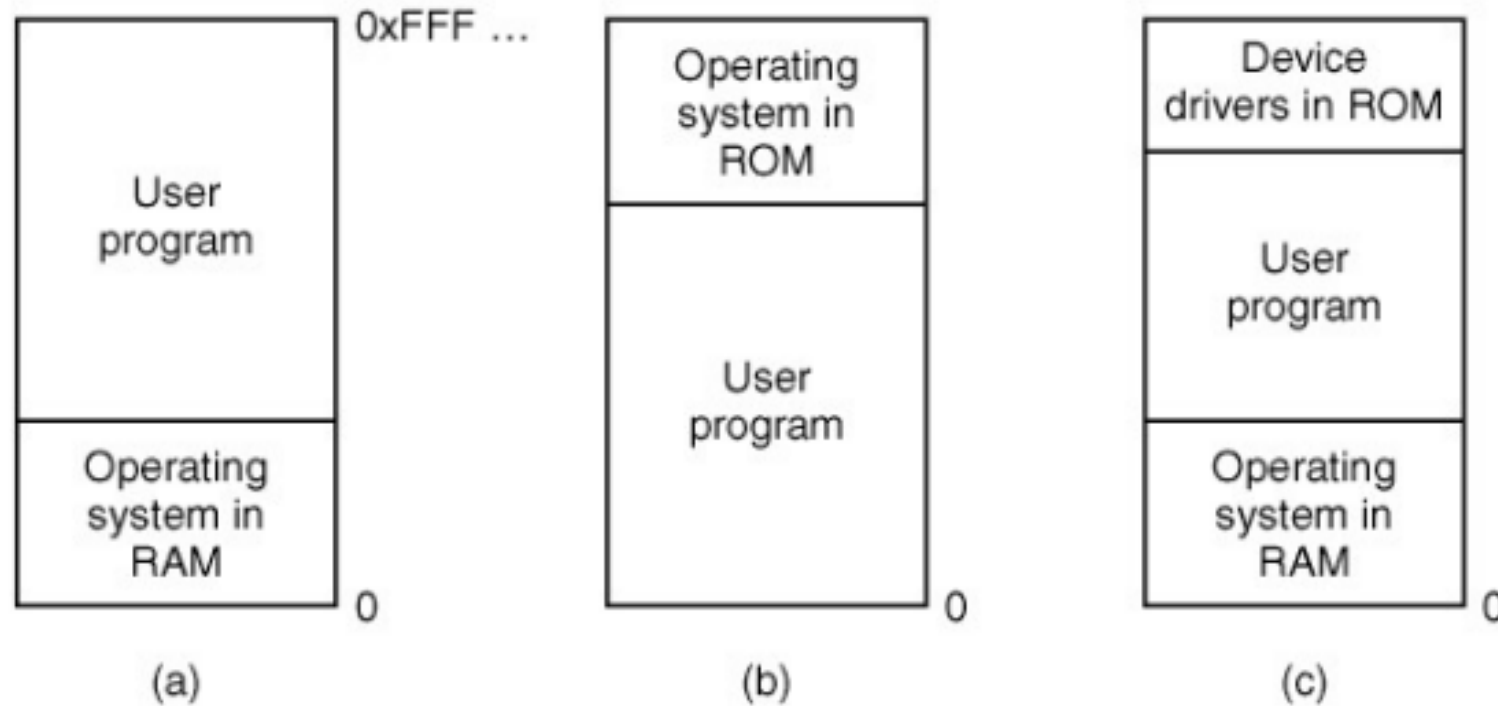
- ❑ Les programmes ont tendance à utiliser de plus en plus de mémoire
 - ❑ La nature a horreur du vide. . .
 - ❑ Les programmes aimeraient souvent utiliser davantage de mémoire que la quantité disponible dans la machine

- ❑ La mémoire appartient aux processus
 - ❑ Il faut faire en sorte que **plusieurs processus puissent co-exister** en ayant accès à différentes parties de la mémoire
 - ❑ Il faut faire en sorte que les **processus ne puissent pas interférer** avec la mémoire des autres processus

- ❑ L'espace mémoire utilisé par un processus peut changer au cours du temps!
- ❑ Opérateur « new » en Java. . .

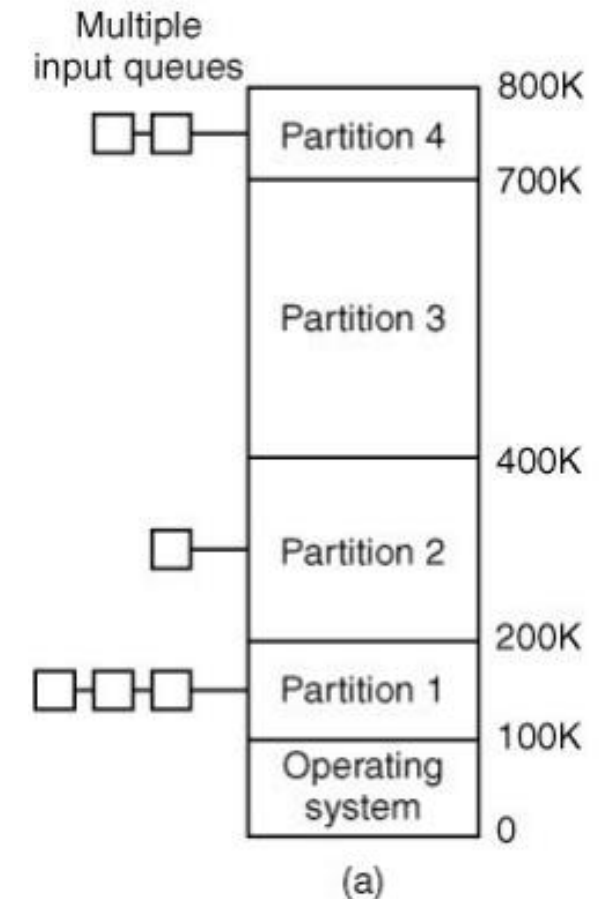
- ❑ Idée : diviser la mémoire en deux parties : OS, application
 - ❑ Par exemple: MS-DOS
- ❑ On peut choisir les adresses mémoire en dur dans les programmes

- ❑ On ne peut exécuter qu'un seul programme à la fois (que se passerait-il sur deux programmes voulaient accéder à la même adresse?)
- ❑ Pas de protection contre les programmes malveillants



- ❑ Idée : **diviser la mémoire en partitions fixes** (éventuellement de tailles différentes)
 - ❑ E.g., IBM OS/360 Exécuter chaque programme dans la partition la plus petite possible
 - ❑ En ré-écrivant les adresses mémoires (« relocation » d'application)
 - ❑ On peut exécuter plusieurs programmes à la fois

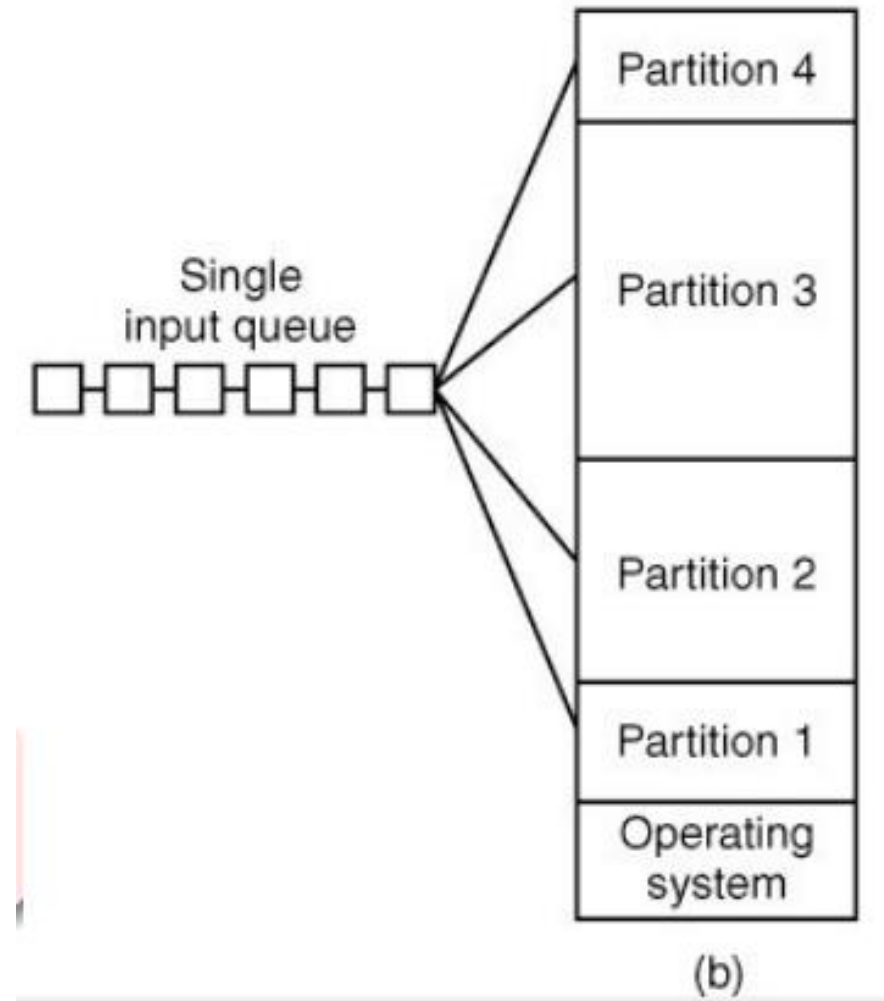
- ❑ Chaque programme doit savoir à l'avance combien de mémoire il va utiliser
- ❑ Pas de protection contre les programmes malveillants
- ❑ Chaque processus doit attendre qu'une des partitions soit libre



❑ Petite optimisation : **utiliser une seule file d'attente**

- ❑ Un peu moins d'attente pour les processus
- ❑ Tous les autres problèmes persistent

On ne va pas résoudre le problème entièrement comme ça...



- ❑ Gérer la mémoire processus par processus est trop grossier
 - ❑ On gaspillerait de la mémoire en conservant de l'espace libre pour chaque processus
 - ❑ Et on ne pourrait « *swapper* » que des processus tout entiers
 - ❑ La pagination gère la mémoire à un grain un peu plus fin (par exemple : 1 page = 4 kB)

Idée #1

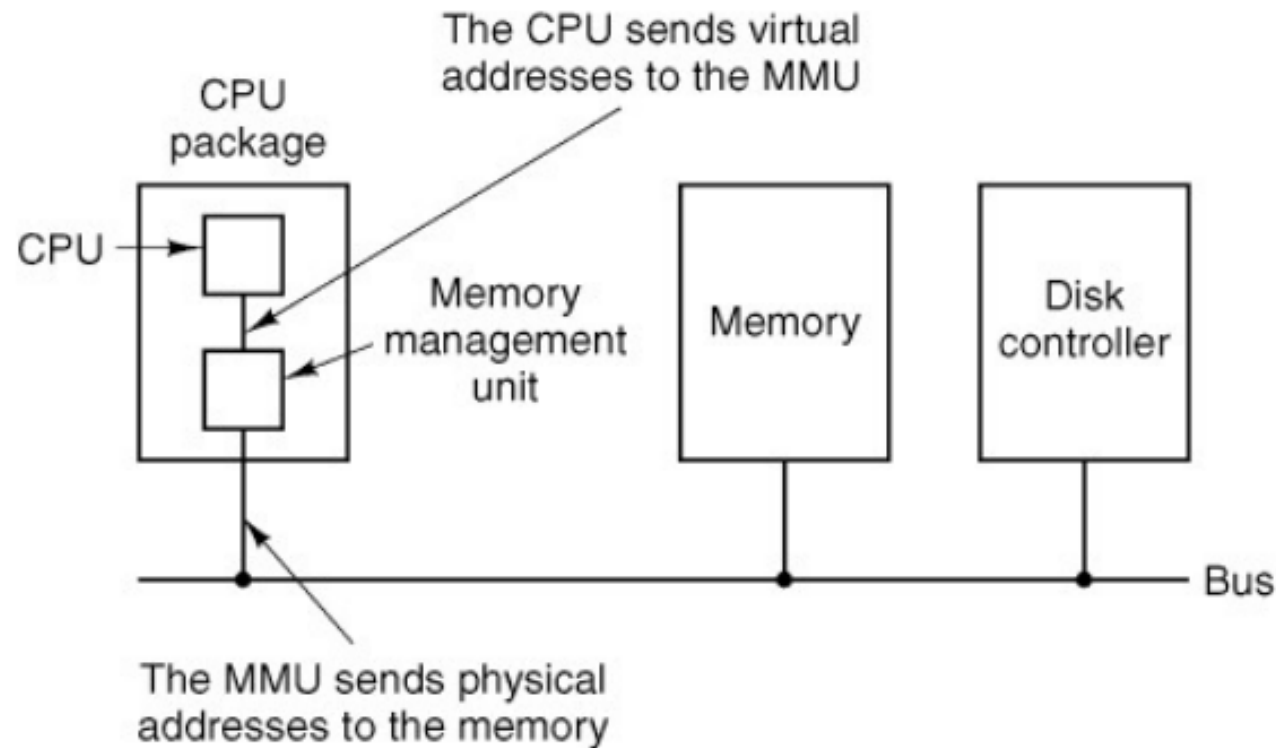
On va **dissocier les adresses mémoire** utilisées par les programmes, et les adresses mémoires au niveau matériel.
Par exemple : **l'adresse logique** 1007 (utilisée par le programme) correspond à **l'adresse physique** 1 000 001 007 (où la variable est vraiment stockée)

Idée #2

On va donner à chaque programme l'illusion qu'il peut utiliser l'espace d'adressage (32-bits ou 64-bits) tout entier

LE “MEMORY MANAGEMENT UNIT”

- ❑ Pour implémenter la pagination on va avoir besoin de support hardware: le Memory Management Unit (MMU)

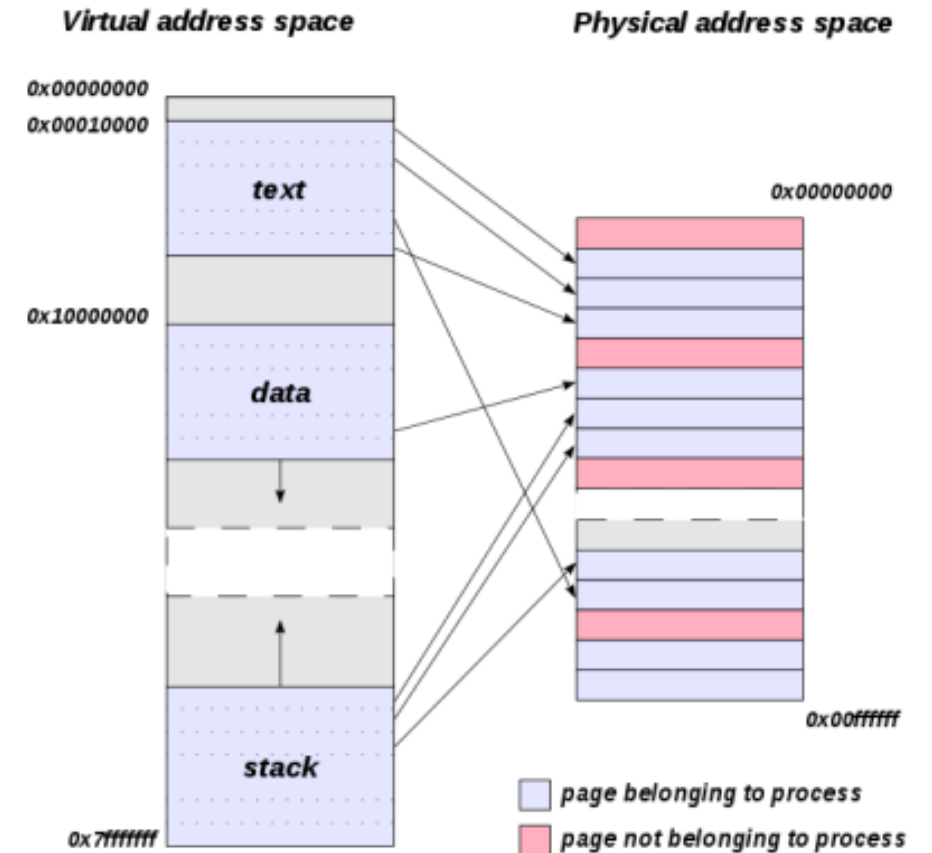


- ❑ Le MMU maintient une **table des pages mémoire** qui contient les correspondances d'adresses logiques/physiques
 - ❑ Quand le programme demande une adresse logique: le MMU la convertit en adresse physique
 - ❑ Si la page n'est pas encore dans la table: le MMU génère un **défaut de page** (redonne la main à l'OS, lui permet de changer les correspondances)

FONCTIONNEMENT DU “MEMORY MANAGEMENT UNIT” 2/2

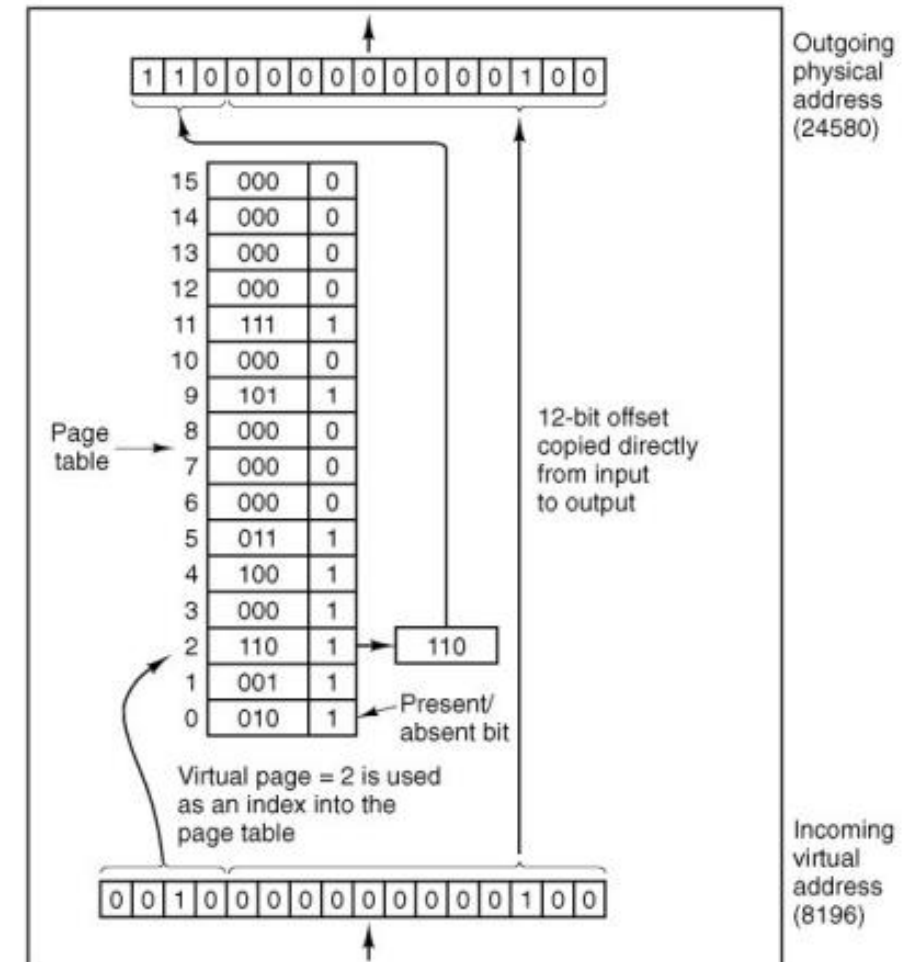
- ❑ Changer les correspondances
nécessite d'être en mode
noyau

Question : pourquoi ne peut-on
pas convertir les adresses en
logiciel ?



IMPLÉMENTATION DE LA PAGINATION

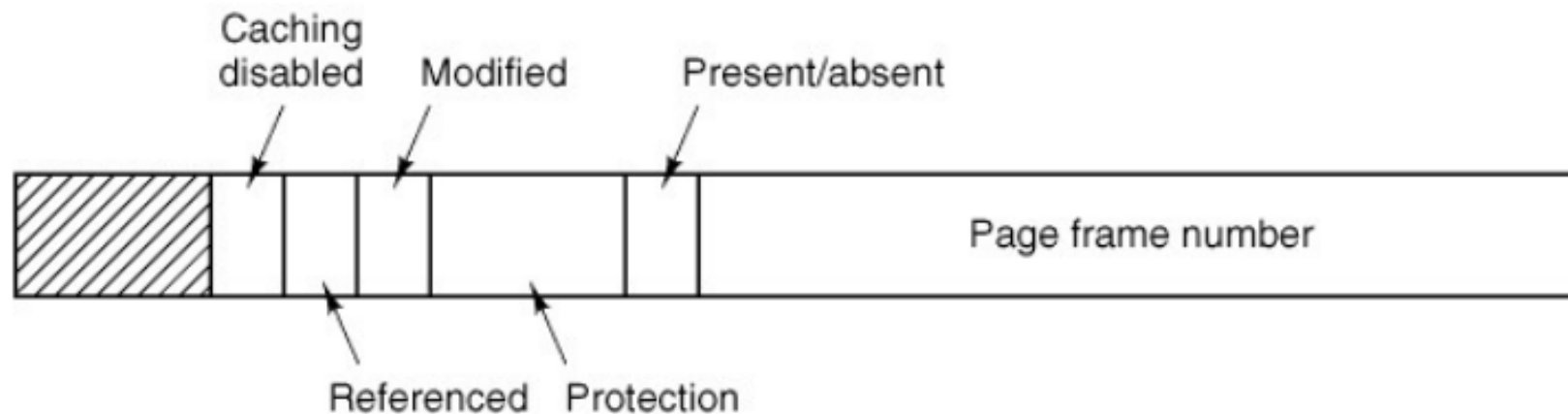
- ❑ La taille des pages doit être une puissance de 2
- ❑ Dans cet exemple: 4096 octets



- ❑ La table des pages contient plusieurs autres informations :
 - ❑ **Présent/absent** : si un processus veut accéder à une page absente, le MMU génère un défaut de page (pour permettre à l'OS d'amener la page en mémoire)
 - ❑ **Protection** : peut-on lire/écrire/exécuter le contenu de cette page ?
 - ❑ **Modified** : si la page n'a pas été modifiée alors on peut la « *swapper* » sans devoir ré-écrire sur disque

STRUCTURE D'UNE ENTRÉE DANS LA TABLE DES PAGES 2/2

- ❑ **Referenced** : si personne n'a utilisé cette page alors c'est une bonne candidate pour se faire swapper Caching
- ❑ **disabled** : utile pour les entrées-sorties memory-mapped (les écritures doivent être effectuées immédiatement)



- ❑ Problème : chaque processus doit maintenir une **table de pages énorme!**
 - ❑ Avec des pages de 4 kB, pour une machine 32-bits il nous faut $2^{28} = 1,048,576$ entrées par processus (et $2^{52} = 4,503,599,627,370,496$ entrées sur une machine 64-bit!)
 - ❑ La table des pages est stockée en mémoire vive

Optimisation #1 : Diviser la table des pages en morceaux, ne stocker en mémoire que les parties effectivement utilisées Ça ne sert à rien de garder des milliards d'entrées "vides"

Optimisation #2 : Le MMU garde un cache des entrées utilisées récemment Le MMU doit rarement aller chercher une entrée en mémoire, la plupart du temps il effectue la conversion en < 1 cycle d'horloge

- ❑ Les entrées de la table de page contiennent des bits de protection
 - ❑ Est-ce que le processus est autorisé à lire/écrire/exécuter les données de cette page mémoire ?
- ❑ L'OS initialise ces permissions et l'ensemble des correspondances d'adresses à chaque changement de contexte Chaque processus ne peut accéder qu'à "son propre espace mémoire"

- ❑ Et si un processus essaye d'accéder à une page mémoire interdite ?
 - ❑ Le MMU génère un défaut de protection
 - ❑ L'OS reprend la main
 - ❑ L'OS génère une erreur de segmentation (segmentation fault)
 - ❑ Cela envoie un signal SIGSEGV au processus
 - ❑ Et en général le signal tue le processus

PLAN

INTRODUCTION

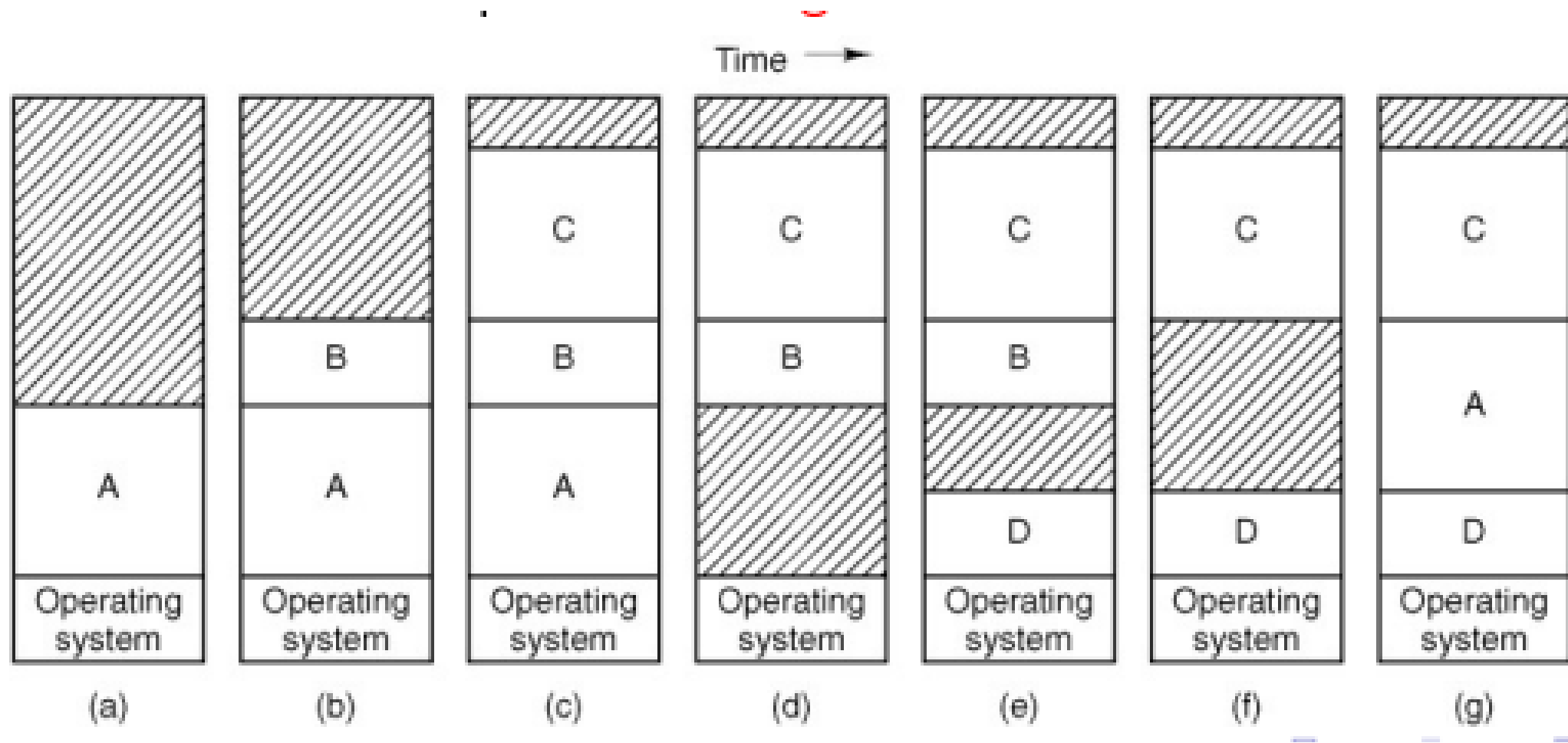
I. MÉMOIRE VIRTUELLE

II. REMPLACEMENT DES PAGES

III. ALLOCATIONS DE MÉMOIRE

CONCLUSION

- ❑ On doit être capable de **déplacer la mémoire d'un processus d'un endroit à l'autre**
 - ❑ Quand la mémoire physique est pleine : **déplacer de la mémoire vers le disque**
 - ❑ Quand le processus doit s'exécuter de nouveau : **ramener la page en mémoire** (peut-être à une autre adresse)
- ❑ Il y a un coût : **les accès disque sont très lents!**
 - ❑ Quand le système passe tout son temps à déplacer des pages entre la mémoire et le disque : **thrashing**



- ❑ Si les processus utilisent plus de mémoire virtuelle que la mémoire physique de la machine, alors il faut “swapper” des pages sur disque
 - ❑ Quelles pages va-t-on swapper en premier ?
- ❑ Chaque fois qu'on peut ramener une page du disque vers la mémoire, il faut choisir une victime qui sera envoyée de la mémoire vers le disque

- ❑ L'algorithme optimal est très simple :
 - ❑ Garder en mémoire les pages mémoires qui vont être utilisées bientôt, envoyer sur disque celles qui seront inutilisées le plus longtemps
 - ❑ Cela retarde le prochain défaut de page aussi loin que possible
 - ❑ Petit souci : comment peut-on deviner quelles pages vont être utilisées dans le futur proche ?

L'ALGORITHME DE REMPLACEMENT LEAST RECENTLY USED (LRU)

- ❑ Algorithme #1 : LRU
 - ❑ On choisit la page qui n'a pas été utilisée depuis le plus longtemps
- ❑ Cet algorithme est très proche de l'optimal
- ❑ Mais : il est très coûteux à implémenter
 - ❑ Il faut garder une trace de tous les accès mémoire
 - ❑ Et trier les pages selon la date du dernier accès
 - ❑ Rappel: on a énormément de pages mémoire
(1 GB de mémoire = 244 000 pages)

L'ALGORITHME DE REMPLACEMENT NOT RECENTLY USED (NRU) 1/2

- ❑ Algorithme #2 : on va relâcher un peu la contrainte du tri
 - ❑ Un peu moins efficace, mais plus facile à implémenter
- ❑ Chaque entrée dans la table des pages a deux bits spéciaux :
referenced et **modified**
 - ❑ Le matériel met ces bits à 1 au moment approprié
 - ❑ Seul l'OS peut les remettre à 0

L'ALGORITHME DE REMPLACEMENT NOT RECENTLY USED (NRU) 2/2

- ❑ Les pages peuvent être classées en 4 catégories :
 1. Not referenced, not modified
 2. Not referenced, modified
 3. Referenced, not modified
 4. Referenced, modified
- ❑ Algorithme : on “swappe” la page de la catégorie la plus basse possible
 - ❑ La performance n'est pas parfaite mais au moins c'est facile à implémenter

QUAND SWAPPER LES PAGES MÉMOIRE ?

- ☐ **Sur demande** : ramener les pages du disque vers la mémoire quand le programme veut y accéder
 - ☐ Simple
 - ☐ Pas très efficace : au démarrage tous les processus auraient énormément de défauts de page
- ☐ **Pre-fetching** : essayer de deviner à l'avance quelles pages vont bien tôt être utilisées
 - ☐ Par exemple : quand on a un défaut de page, on ramène en mémoire la page demandée **et aussi les N pages suivantes**
 - ☐ Cela fonctionne grâce aux propriétés de localité spatiale

QUAND SWAPPER LES PAGES MÉMOIRE ?

- ❑ **Pre-cleaning** : copier les pages modifiées sur disque proactivement, mais sans les swapper
 - ❑ Si la page se fait swapper sans être modifiée de nouveau : on a gagné en performance
 - ❑ Sinon: on a gaspillé de la capacité sur le bus

PLAN

INTRODUCTION

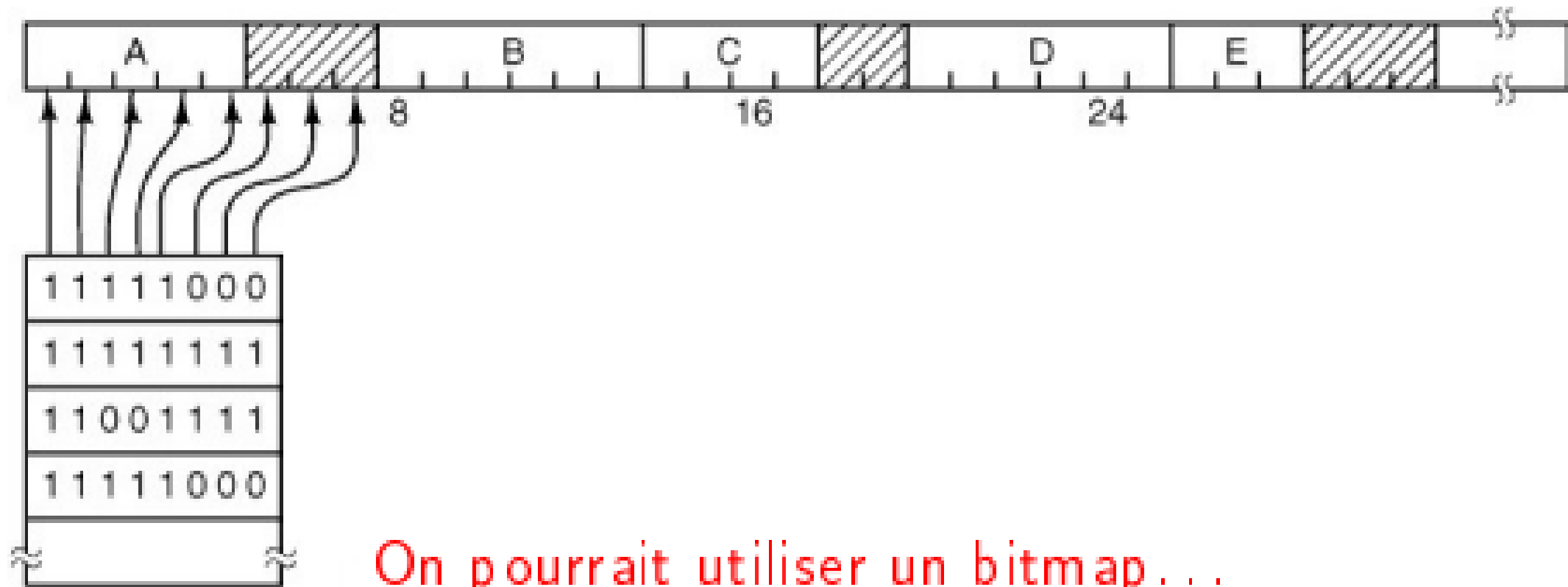
I. MÉMOIRE VIRTUELLE

II. REMPLACEMENT DES PAGES

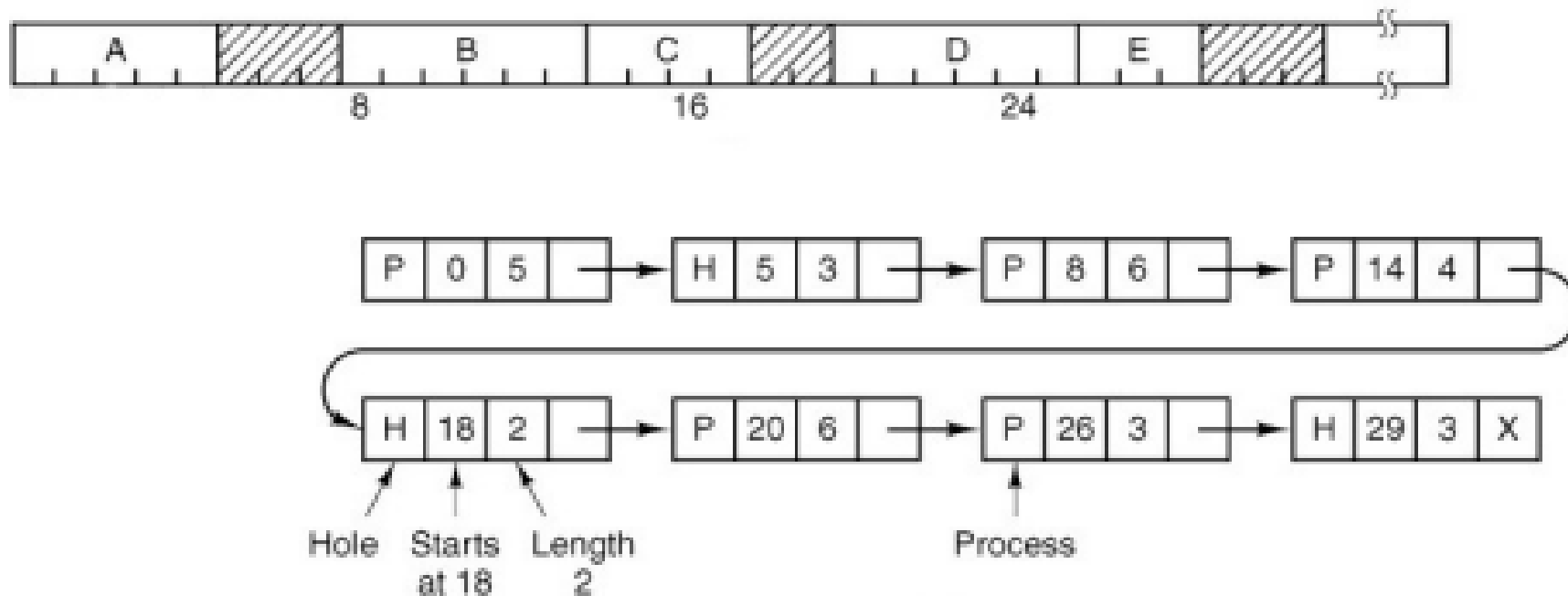
III. ALLOCATIONS DE MÉMOIRE

CONCLUSION

- ❑ Les variables locales et les paramètres de fonctions sont stockées **sur la pile**
- ❑ Mais où stocke-t-on les variables comme **x = new Object();** ?
 - ❑ On les stocke **sur le tas (heap)**
- ❑ L'opérateur new doit trouver un espace libre dans la mémoire pour stocker ce nouvel objet
 - ❑ Et il faut conserver l'information de **quelle partie de mémoire est actuellement utilisée ou non**

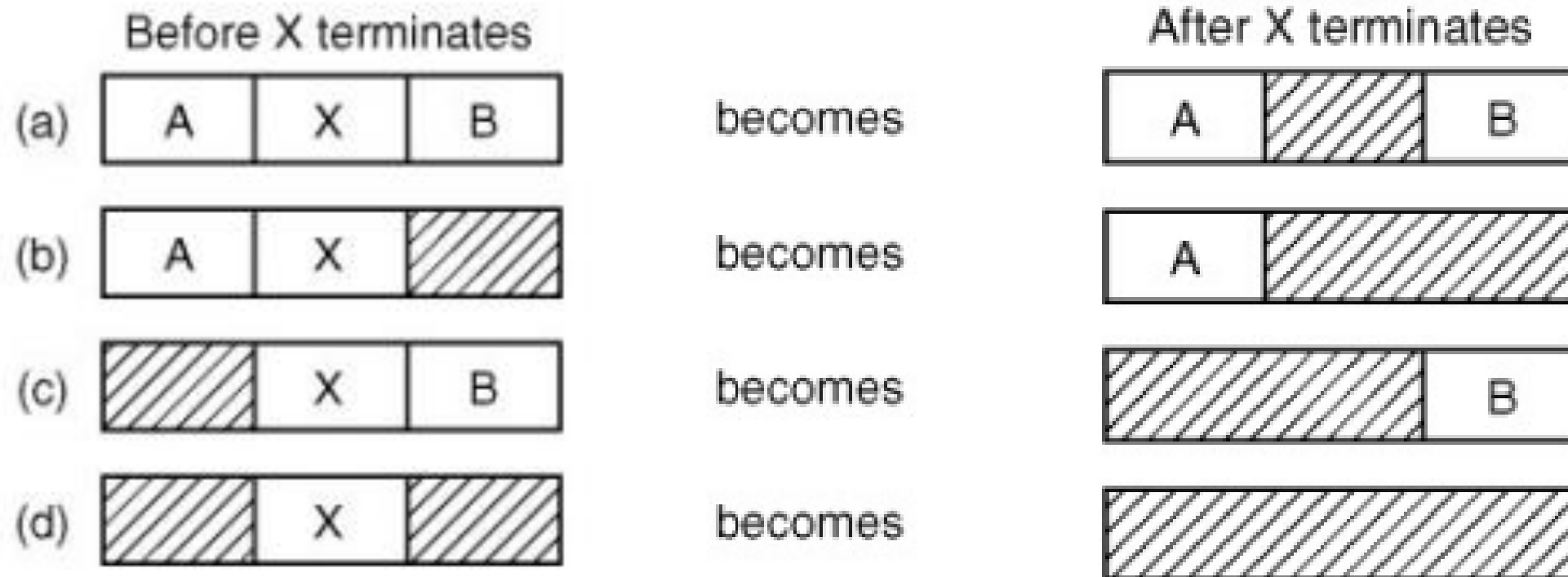


- ❑ Problème des bitmaps : **chercher un espace libre** est très coûteux
- ❑ Une meilleure solution : **liste chaînée** de trous (Holes) et de zones utilisées par le **P**rocessus



- ❑ Problème : **fragmentation**

- ❑ Avec les listes chaînées on peut fusionner les espaces libres contigus



☐ First fit/Next fit

- ☐ First fit: prendre le premier trou suffisamment grand dans la mémoire

- ☐ Next fit : démarrer au dernier bloc alloué

- ☐ Simple

- ☐ Problème : fragmentation mémoire

☐ Best fit : trouver le plus petit trou possible parmi les trous assez grands

- ☐ Problème : c'est coûteux

- ❑ **Quick fit** : créer des réservoirs de zones mémoire de différentes tailles (par exemple : 16 B, 128 B, 1 kB, 20 kB, 1 MB)
 - ❑ Allouer un espace de la taille immédiatement supérieure
 - ❑ Problème : on gaspille de la mémoire si le bloc alloué est trop grand
 - ❑ Problème : combien faut-il créer de blocs de chaque taille ?
- ❑ **Buddy fit** :
 - ❑ Les espaces mémoire ont toujours une taille en puissance de 2
 - ❑ Si un bloc alloué est plus que 2 fois trop grand : le diviser en 2
 - ❑ Si deux blocs “amis” sont libres : les fusionner

En conclusion, nous avons parcouru les principaux concepts liés à la gestion de la mémoire dans les systèmes informatiques. Nous avons commencé par explorer l'importance de la mémoire virtuelle en permettant aux systèmes de gérer efficacement les ressources physiques et d'optimiser l'utilisation de la RAM. Nous avons examiné les mécanismes de remplacement des pages, qui jouent un rôle crucial dans la gestion de la mémoire virtuelle en choisissant quelles pages doivent être déplacées entre la mémoire principale et le stockage de masse.

De plus, nous avons étudié les stratégies d'allocation de mémoire pour comprendre comment les programmes obtiennent de l'espace dans la mémoire virtuelle. En combinant ces connaissances, les étudiants sont désormais mieux préparés à comprendre les subtilités de la gestion de la mémoire dans les systèmes informatiques modernes et à développer des compétences essentielles pour optimiser les performances des applications et des systèmes.